

Guía de estudio para el escrito

Ingeniería de software, SCM y Git

Resumen de Sommerville, capítulos 1 y 2, y Pro Git, capítulos 1, 2 y 3. Incluye diagramas originales, resolución de los dos formularios, preguntas de repaso y práctica resuelta.

Cómo se preparó. Primero se analizaron los 24 ítems de las cuatro capturas: definición y alcance del software; actividades del proceso y de apoyo; modelos y prototipos; SCM; naturaleza distribuida de Git; estados, configuración y comandos. Luego se amplió la cobertura a los demás apartados de los cinco capítulos.

Alcance de las preguntas. El banco propone variantes razonables a partir de los contenidos y del estilo observado. No es una predicción del examen ni puede garantizar todas las redacciones posibles. Estudiá la explicación: memorizar solamente letras no sirve si cambian las opciones.

Orden de estudio sugerido. Leé primero la teoría, practicá los comandos sin mirar las soluciones y terminá con el banco de preguntas. Volvé a los conceptos en los que te equivoques. En los formularios, prestá especial atención a las palabras NO, únicamente, siempre y previamente.

Convenciones. Los ejemplos usan la rama `master`, como el libro. Si tu repositorio usa `main`, sustituí el nombre. Las páginas de las fuentes son las impresas en el libro, no las del visor PDF. El PDF de Pro Git aportado es una traducción de la segunda edición actualizada en 2021, aunque el programa de la materia mencione 2014.

Material visual. Los recortes proceden de los PDF que aportaste. Se conserva la atribución de cada figura y se agrega una lectura guiada. Los ejemplos y preguntas adicionales son elaborados para esta guía.

Mapa del documento. Teoría de ingeniería de software; procesos; SCM; fundamentos y práctica de Git; respuestas de las capturas; banco de 100 preguntas; ejercicios integradores; repaso final y fuentes.

1. Qué revelan los formularios

Prioridades deducidas de las cuatro capturas

Definiciones precisas. Preguntan qué es software, qué es ingeniería de software y qué NO es una definición adecuada. La respuesta completa debe incluir documentación, configuración y el ciclo de vida, además del código.

Diferencias entre actividades. Se confunden especificación con validación, y actividades fundamentales con actividades de apoyo. Aprendé el propósito de cada una: definir lo necesario, construirlo, comprobarlo y adaptarlo.

Elección de procesos. Aparecen la utilidad de los prototipos y las ventajas del desarrollo incremental. Es razonable practicar también cascada, reutilización, espiral y RUP, porque están en el capítulo 2.

SCM. Se evalúan la identificación y el control del software, los elementos de configuración y las actividades de gestión. Git implementa principalmente control de versiones: SCM tiene un alcance mayor.

Git conceptual y operativo. Las capturas preguntan por `init`, `add .`, `log`, `push`, identidad del autor, archivos untracked, línea de comandos y trabajo sin conexión. Para poder responder variantes, hay que entender qué cambia cada comando en el directorio de trabajo, el índice, el repositorio local y el remoto.

Confusiones que conviene corregir primero

- La crisis del software se vincula con la conferencia de 1968; no con la revolución de la informática personal.
- Especificar es definir servicios y restricciones; validar es comprobar que se satisface lo requerido.
- Un prototipo permite aprender; no es automáticamente el producto final.
- El desarrollo incremental facilita cambios; no garantiza una estructura que nunca necesite refactorización.
- Un commit necesita identidad de autor disponible; no necesita una URL remota.
- Git es distribuido y permite commits y ramas locales sin Internet.
- `add` prepara, `commit` registra localmente y `push` comparte commits con otro repositorio.

Lo que no muestran las fotos. No hay una consigna práctica completa. Los ejercicios incluidos entrenan los procedimientos de estos capítulos; no se presentan como una reconstrucción del práctico del docente.

2. Software e ingeniería de software

Sommerville, capítulo 1: conceptos centrales

Software es el conjunto de programas junto con la documentación y los datos de configuración necesarios para operarlos y mantenerlos. Puede incluir varios programas, bibliotecas, instrucciones de instalación, manuales, documentación del diseño y sitios de soporte. Un ejecutable aislado no describe todo el producto.

El software es abstracto e intangible. No está sujeto a las mismas restricciones de materiales y fabricación que el hardware. Eso permite una enorme flexibilidad, pero también que alcance gran complejidad y sea difícil y costoso de modificar. No se desgasta físicamente por ejecutarse; los cambios mal gestionados pueden deteriorar su estructura.

Ingeniería de software es una disciplina de ingeniería que abarca todos los aspectos de la producción de software, desde la especificación inicial hasta el mantenimiento en operación. Aplica métodos, herramientas y conocimientos de manera sistemática, atendiendo calidad, tiempo, costo y restricciones organizacionales. Incluye gestión y actividades técnicas.

Programar es una parte. También hay que comprender necesidades, diseñar, probar, documentar, gestionar versiones, desplegar y evolucionar. La producción profesional suele involucrar equipos y personas distintas para desarrollar, usar y mantener.

Productos y disciplinas relacionadas

- **Producto genérico:** se ofrece a un mercado; el proveedor controla principalmente la especificación. Ejemplo: un editor de texto comercial.
- **Producto a medida:** se encarga para un cliente particular, que normalmente controla la especificación. Ejemplo: un sistema específico para una organización.
- **Caso mixto:** un producto genérico se configura o adapta para una empresa, como un ERP.
- **Ciencias de la computación:** se centran en teoría y fundamentos. La ingeniería de software aborda la producción práctica bajo restricciones.
- **Ingeniería de sistemas:** abarca el sistema completo, incluidos hardware, software, procesos y su integración. La ingeniería de software forma parte de ese campo más amplio.

Origen histórico. La conferencia de la OTAN de 1968 discutió la crisis del software: proyectos tardíos, costosos, poco confiables y que no satisfacían necesidades. Los enfoques individuales no escalaban a sistemas grandes. La fecha esperada en el formulario es 1968.

Costos. Sommerville menciona como orientación alrededor de 60% de desarrollo y 40% de pruebas, no como ley universal. En sistemas a medida de larga vida, la evolución puede costar más que el desarrollo inicial.

Fuente: Sommerville 9.ª ed., pp. 4-10. El recuadro histórico del capítulo 1 dice 1968, aunque el prefacio menciona 1969.

3. Calidad, diversidad y fundamentos

Los cuatro grupos de atributos esenciales

Mantenibilidad. El software debe poder modificarse para atender nuevas necesidades. Importan su estructura, comprensibilidad y documentación. No significa solamente corregir errores.

Confiabilidad y seguridad. Debe funcionar de forma confiable, evitar daños y protegerse de accesos o modificaciones no autorizados. Conviene distinguir evitar accidentes de resistir ataques: ambas preocupaciones pueden ser relevantes.

Eficiencia. Debe aprovechar razonablemente memoria, procesamiento y otros recursos. Incluye tiempos de respuesta y capacidad de procesamiento; no se reduce a que el código sea corto.

Aceptabilidad. Debe ser comprensible, utilizable y compatible con el contexto y los otros sistemas de sus usuarios. Un sistema técnicamente correcto puede fracasar si las personas no pueden usarlo.

Tipos de aplicaciones: reconocer ejemplos

- **Independientes:** funcionan localmente sin necesitar servicios de red, como cierto software de edición.
- **Interactivas basadas en transacciones:** procesan interacciones y actualizaciones de datos, como compras por Internet.
- **Control embebido:** controla dispositivos, como un sistema de frenado.
- **Procesamiento por lotes:** procesa conjuntos de entradas, como una liquidación mensual de sueldos.
- **Entretenimiento:** juegos y otras aplicaciones centradas en la experiencia de uso.
- **Modelado y simulación:** representa procesos científicos o de ingeniería, a menudo con mucho cálculo.
- **Adquisición de datos:** recoge información de sensores y la envía o procesa.
- **Sistemas de sistemas:** integra sistemas que pueden tener desarrollos y responsabilidades propios.

No existe un método universalmente mejor. Un sistema crítico exige evidencia de seguridad y análisis riguroso; una aplicación con requisitos inciertos se beneficia de retroalimentación temprana. Los límites entre categorías pueden superponerse.

Fundamentos comunes: proceso comprendido y gestionado; confiabilidad y desempeño; comprensión y gestión de requisitos; uso eficaz de recursos y reutilización cuando corresponda.

Retos generales: heterogeneidad de plataformas y sistemas heredados; cambios empresariales y sociales y necesidad de entrega rápida; confianza y seguridad.

Efecto de la Web. Favoreció sistemas distribuidos, servicios, reutilización e instalación centralizada de actualizaciones. Los requisitos suelen evolucionar y la entrega incremental resulta útil. Las observaciones del libro sobre limitaciones de navegadores corresponden a su época; el principio importante es adaptar el desarrollo al entorno de ejecución.

Fuente: Sommerville, pp. 7-14.

4. Ética y estudios de caso

Responsabilidad profesional

Confidencialidad: proteger información de clientes y empleadores, incluso cuando no exista un acuerdo formal. **Competencia:** representar honestamente las capacidades propias y reconocer límites. **Propiedad intelectual:** respetar los derechos sobre código, datos y documentos. **Uso adecuado de computadoras:** no abusar de accesos o conocimientos técnicos.

La responsabilidad no termina en obedecer una orden ni en aplicar una habilidad técnica. Puede haber conflictos entre confidencialidad, interés del empleador y seguridad pública. En un caso de examen hay que identificar afectados, consecuencias, obligaciones y alternativas; no responder que toda decisión se resuelve con una única regla automática.

Ocho principios del código ACM/IEEE presentado en el libro

- **Público:** priorizar el interés público.
- **Cliente y empleador:** atender sus intereses de forma compatible con el interés público.
- **Producto:** procurar estándares profesionales de calidad.
- **Juicio:** mantener integridad e independencia profesional.
- **Gestión:** promover una administración ética del desarrollo y mantenimiento.
- **Profesión:** contribuir a su integridad y reputación.
- **Colegas:** actuar justamente y brindar apoyo.
- **Uno mismo:** aprender continuamente y mejorar la práctica profesional.

Los tres casos del capítulo 1

Bomba de insulina. Sistema embebido que recibe mediciones, calcula una dosis y controla una bomba. Su ejemplo enseña disponibilidad, confiabilidad y seguridad: debe actuar cuando corresponde y entregar la cantidad correcta. Una falla puede causar daño físico.

MHC-PMS, información de pacientes de salud mental. Gestiona registros, seguimiento y reportes. Se necesitan privacidad, integridad y disponibilidad. Mantener copias locales permite trabajar sin conexión, pero aumenta la dificultad de proteger la información. Es un ejemplo de compromisos entre atributos.

Estación meteorológica remota. Recoge y resume datos de sensores, transmite información y administra recursos limitados. También monitorea fallas y energía y admite mantenimiento remoto. Se relaciona con sistemas de adquisición de datos y con sistemas de sistemas.

Cómo contestar un caso nuevo. Identificá el tipo de sistema, el atributo de calidad prioritario, las consecuencias de una falla y el enfoque de desarrollo que lo atiende. Justificá la elección con condiciones concretas.

Fuente: Sommerville, pp. 14-23. Síntesis de los principios presentados en el libro.

5. Procesos: lo que nunca debe confundirse

Sommerville, capítulo 2

Un **proceso de software** es un conjunto de actividades relacionadas que conduce a producir y evolucionar software. Un **modelo de proceso** es una representación simplificada del proceso desde una perspectiva; no contiene todos los detalles ni es el producto que se construye.

Las cuatro actividades fundamentales son:

- **Especificación:** acordar servicios y restricciones. Pregunta: ¿qué se necesita?
- **Desarrollo:** diseñar e implementar. Pregunta: ¿cómo se construye?
- **Validación:** comprobar conformidad y necesidades reales. Pregunta: ¿sirve y cumple lo requerido?
- **Evolución:** modificar para necesidades cambiantes. Pregunta: ¿cómo se mantiene útil?

La lista «codificación, compilación, depuración y despliegue» contiene tareas reales, pero no es la clasificación fundamental de Sommerville.

Qué describe un proceso

Actividades y orden, **productos** resultantes, **roles** responsables, **precondiciones** que deben cumplirse antes y **postcondiciones** que deben cumplirse después. Ejemplo: una actividad de revisión produce observaciones; un revisor tiene la responsabilidad de evaluarlas; su resultado debe quedar registrado.

Actividades de apoyo. En el formulario, la terna esperada es gestión de proyectos, gestión de configuración y aseguramiento de calidad. Acompañan el ciclo de vida. Especificación, diseño y pruebas son actividades técnicas; que sean importantes no las convierte en la terna de apoyo de esa pregunta.

Dirigido por un plan y ágil

En un proceso **dirigido por un plan**, las actividades se planifican por anticipado y el avance se compara con el plan. En un proceso **ágil**, la planificación se realiza incrementalmente y se facilita la adaptación. Ágil no equivale a ausencia de planificación, diseño o pruebas.

Los enfoques se pueden combinar. Incremental no significa automáticamente ágil: los incrementos también pueden planificarse anticipadamente. No hay un proceso ideal para todos los proyectos.

Estandarización. Reducir variaciones innecesarias entre procesos de una organización puede mejorar comunicación, capacitación y soporte de herramientas. No implica aplicar ciegamente el mismo proceso a todo sistema.

Herramientas CASE e IDE. Apoyan modelado, edición, generación de código, compilación, pruebas y depuración. Automatizan tareas y organizan información; no sustituyen el juicio de las personas.

Fuente: Sommerville, pp. 28-29 y 36-40; clasificación de apoyo del formulario.

6. Cascada frente a incremental

Cascada: fases con resultados definidos

Sus fases son requisitos; diseño del sistema y software; implementación y pruebas de unidad; integración y pruebas del sistema; operación y mantenimiento. Favorece documentación y seguimiento del avance. Es apropiada cuando los requisitos se comprenden y son relativamente estables, o se necesita mucha coordinación y evidencia.

Su limitación es el costo de cambiar decisiones tempranas. **No significa que nunca haya retroalimentación:** en la práctica se vuelve a fases anteriores, pero hacerlo puede ser costoso.

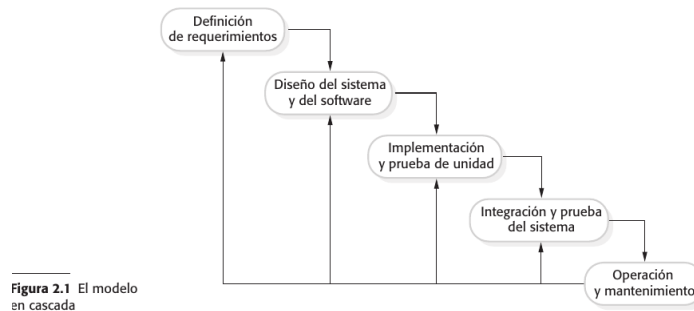


Figura 2.1, Sommerville, p. 30. Lee los bloques de arriba abajo: cada fase produce resultados que alimentan la siguiente. Las conexiones de regreso representan retroalimentación.

Incremental: versiones que agregan valor

Se parte de una implementación inicial y se incorporan funciones mediante versiones. Especificación, desarrollo y validación se entrelazan. Se prioriza lo más importante y se aprende de la respuesta del cliente.

Ventajas: menor costo de acomodar cambios, retroalimentación más sencilla sobre software visible y entrega temprana de funcionalidad útil. **Problemas:** menor visibilidad documental del avance y degradación de la estructura si no se refactoriza; los sistemas grandes requieren atención a la arquitectura y la coordinación.

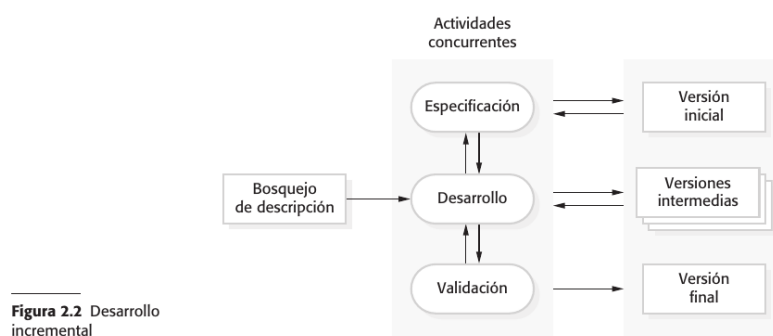


Figura 2.2, Sommerville, p. 33. Las actividades concurrentes alimentan una versión inicial, versiones intermedias y una final. No son fases aisladas que se ejecutan una sola vez.

Para el formulario: si preguntan la ventaja del incremental frente a cascada, elegí incorporar cambios a un costo relativamente bajo. «Requisitos estables desde el principio» caracteriza un escenario favorable a cascada.

7. Reutilización y elección del modelo

Ingeniería orientada a la reutilización

Construye el sistema aprovechando componentes, servicios o productos existentes. No consiste simplemente en copiar código: hay que buscar, evaluar, adaptar e integrar piezas y desarrollar lo que falte.

Secuencia del modelo: especificación inicial; análisis de componentes; modificación de requisitos; diseño con reutilización; desarrollo e integración; validación.

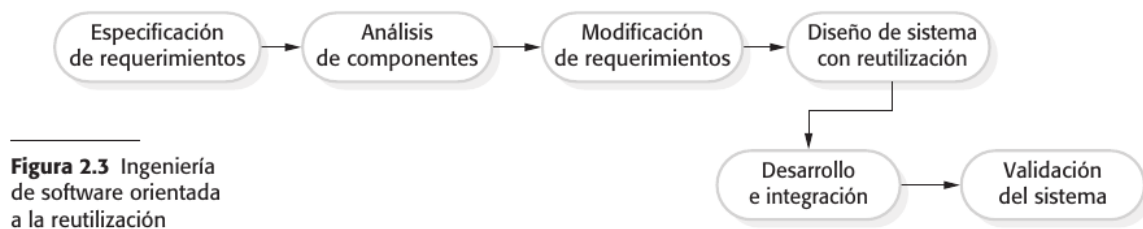


Figura 2.3, Sommerville, p. 35. Observá que los requisitos se revisan después de buscar componentes. Las capacidades disponibles pueden obligar a negociar lo solicitado.

Qué se reutiliza: servicios invocables remotamente; colecciones de objetos o componentes; sistemas completos configurables, incluidos productos comerciales COTS.

Ventajas: menos desarrollo nuevo, menores costos y riesgos y posibles entregas más rápidas.

Limitaciones: compromisos entre requisitos y componentes disponibles; integración compleja; menor control sobre la evolución de componentes de terceros. Reutilizar no elimina las pruebas del sistema integrado.

Comparación aplicada

- Requisitos claros, pocas variaciones y necesidad de documentación: considerar un proceso dirigido por un plan, como cascada.
- Requisitos inciertos y necesidad de aprender del usuario: considerar desarrollo incremental y prototipos.
- Funcionalidad estándar ya disponible: considerar reutilización, evaluando ajuste e integración.
- Riesgos técnicos o de negocio significativos: un enfoque dirigido por riesgos, como espiral, ayuda a decidir qué resolver primero.
- Sistema grande: combinar enfoques puede ser razonable; una arquitectura planificada puede sostener subsistemas desarrollados incrementalmente.

Iterativo e incremental. Iterar es repetir actividades para revisar y mejorar; incrementar es agregar funcionalidad. Un proceso puede hacer ambas cosas.

Desarrollo formal, mención del capítulo. Parte de una especificación matemática y aplica transformaciones que preservan propiedades. Puede ser útil en sistemas críticos, pero requiere especialistas. Que una implementación respete una especificación no garantiza que la especificación represente las necesidades correctas.

Cleanroom. El recuadro describe desarrollo incremental con especificación y razonamiento formal; sus pruebas de sistema se enfocan en fiabilidad. Es una variante específica, no la regla general de pruebas de todo proyecto.

Fuente: Sommerville, pp. 29-36.

8. Especificar, diseñar e implementar

Ingeniería de requisitos

Busca comprender y definir servicios y restricciones de operación y desarrollo. Sus errores se propagan al diseño y al código.

- **Factibilidad:** valorar viabilidad técnica, utilidad empresarial, costos y restricciones para decidir si continuar.
- **Obtención y análisis:** conversar, observar tareas, estudiar sistemas existentes, modelar y explorar necesidades.
- **Especificación:** registrar los requisitos de manera acordada. Los de usuario son de alto nivel; los de sistema son más detallados.
- **Validación de requisitos:** comprobar que sean realistas, completos y coherentes. Puede llevar a corregirlos.

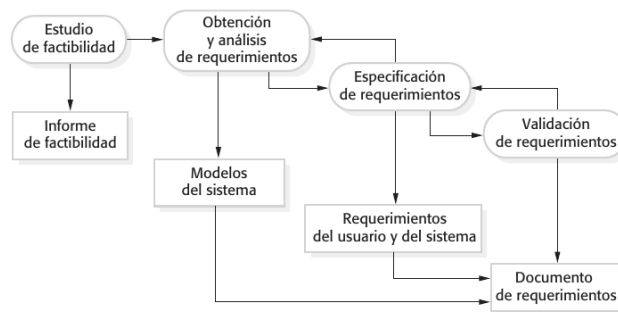


Figura 2.4 Proceso de ingeniería de requerimientos

Figura 2.4, Sommerville, p. 38. Diferenciá actividades de resultados: informe de factibilidad, modelos y documento de requisitos. Las flechas no impiden iteraciones.

Diseño e implementación

El diseño define estructura, datos, componentes e interfaces; la implementación los convierte en un sistema ejecutable. Se retroalimentan y no siempre están completamente separados.

- **Arquitectura:** componentes principales, relaciones y distribución.
- **Interfaces:** contratos de interacción entre componentes; no solamente pantallas.
- **Componentes:** funcionamiento interno de cada parte.
- **Base de datos:** estructuras de datos y su representación persistente, cuando corresponda.

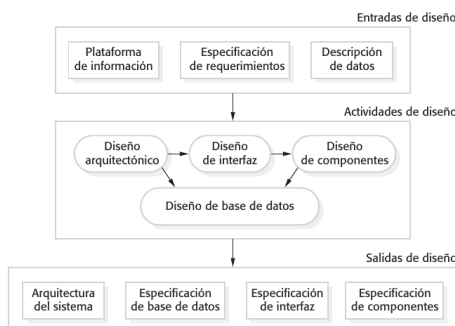


Figura 2.5 Modelo general del proceso de diseño

Figura 2.5, Sommerville, p. 39. Entradas arriba, actividades en el centro y productos abajo. Los requisitos, la plataforma y los datos condicionan las decisiones de diseño.

Ejemplo: «permitir reservar una consulta» es un requisito; decidir módulos y contratos es diseño; escribir su código es implementación; comprobar la reserva es validación.

9. Validación, pruebas y evolución

Verificación y validación

Verificación: comprobar conformidad con lo especificado. **Validación:** comprobar que se satisfacen las necesidades reales. El capítulo reúne ambas bajo V&V: un sistema debe cumplir tanto su especificación como las expectativas del cliente.

Un sistema puede cumplir una especificación equivocada y aun así no servir. La validación no equivale a escribir código ni a diseñar la arquitectura.

No todo es ejecutar pruebas. Revisiones e inspecciones de requisitos, diseños y código también aportan evidencia. Las pruebas ejecutan el programa con datos y comparan resultados; pueden revelar defectos, pero no prueban en general su ausencia absoluta.

Niveles de pruebas

- **Componentes o unidades:** partes individuales, como funciones o clases.
- **Sistema:** componentes integrados, sus interacciones y propiedades funcionales y no funcionales del conjunto.
- **Aceptación:** evaluación con participación y datos del cliente para determinar si el sistema resulta aceptable para su uso.

Pruebas alfa y beta. El texto relaciona alfa con la aceptación de sistemas a medida; beta expone un producto a usuarios potenciales en uso real para recibir informes de problemas antes de una liberación más amplia.

Modelo V. Relaciona niveles de especificación y diseño con planes de prueba: requisitos con aceptación, especificación del sistema con integración del sistema y diseño con integración de subsistemas. La planificación de pruebas puede empezar antes de programar.

Probar y depurar no son sinónimos. Probar detecta comportamientos incorrectos; depurar localiza sus causas y las corrige. Después de corregir se vuelve a probar, incluyendo efectos sobre comportamiento existente.

Evolución del software

El sistema se modifica porque cambian necesidades, negocio, tecnología y entorno, o aparecen errores. Desarrollo y mantenimiento pueden verse como un continuo. Mantener no es solamente reparar: también adaptar y ampliar.

Refactorización: mejora de la estructura interna conservando el comportamiento observable. Ayuda a que futuros cambios sean manejables; no es una garantía de que el software jamás se degrade.

Caso típico: la aplicación pasó todas las pruebas de unidad, pero los módulos intercambian datos incompatibles. Faltan pruebas de integración/sistema: el éxito aislado de las unidades no asegura el éxito conjunto.

Fuente: Sommerville, pp. 40-44.

10. Prototipos y entrega incremental

Anticipar y tolerar cambios

Anticipar o evitar cambios costosos significa aprender antes de comprometer mucho trabajo. **Tolerarlos** significa organizar el proceso y la estructura para incorporarlos a un costo razonable. Ningún enfoque elimina todos los cambios.

Un **prototipo** es una representación inicial para demostrar conceptos, explorar opciones y aclarar requisitos. Puede ser código ejecutable, pantallas o incluso papel. Un prototipo de Mago de Oz simula funcionalidad mediante una persona detrás de la interfaz.

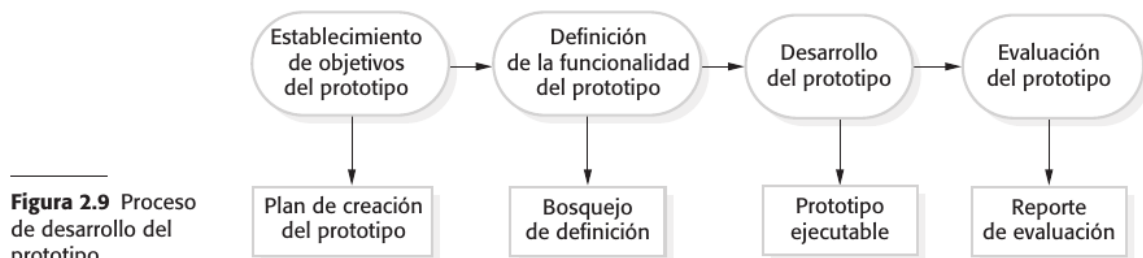


Figura 2.9 Proceso de desarrollo del prototipo

Figura 2.9, Sommerville, p. 45. Primero se define qué se quiere aprender; luego qué incluir, se construye y finalmente se evalúa. Sin objetivos claros, el resultado puede interpretarse mal.

Usos: descubrir requisitos omitidos, validar ideas con usuarios, explorar interfaces y comprobar viabilidad técnica. **Límites:** puede omitir rendimiento, seguridad, tratamiento de errores, documentación y estándares de producción.

Un prototipo desechable no debe convertirse automáticamente en el sistema final: su estructura y calidad pueden no ser suficientes. Además, usuarios o condiciones de evaluación poco representativos pueden llevar a conclusiones equivocadas.

Desarrollo incremental no es lo mismo que entrega incremental

Se puede desarrollar por incrementos y mostrar versiones sin ponerlas en producción. En la **entrega incremental**, partes útiles se instalan y usan en el entorno real. Se entregan primero servicios prioritarios; los requisitos de incrementos futuros pueden refinarse mientras se construye el actual.

Beneficios: valor temprano, experiencia real, cambios más manejables y más oportunidades de probar las funciones prioritarias. **Dificultades:** reconocer infraestructura común, reemplazar un sistema cuyo conjunto de funciones se necesita desde el primer día y trabajar con contratos que exigen una especificación completa anticipada.

Ejemplo: una pantalla simulada de inscripción es un prototipo; un módulo de inscripción que ya usan estudiantes es una entrega operativa, aunque falten reportes y pagos.

Detalle de la fuente: en p. 47 aparece una frase que dice que los clientes «deben esperar» la entrega completa; contradice el beneficio y la explicación inmediata. La idea correcta en ese contexto es que NO necesitan esperar todo el sistema para obtener valor.

Fuente: Sommerville, pp. 43-48.

11. Espiral y Proceso Unificado Racional

Espiral de Boehm: el riesgo dirige el proceso

Cada vuelta incluye establecer objetivos, alternativas y restricciones; evaluar y reducir riesgos; desarrollar y validar; revisar y planificar la siguiente vuelta. La técnica concreta se selecciona según el riesgo: un prototipo puede aclarar una interfaz; un análisis formal puede atender propiedades críticas.

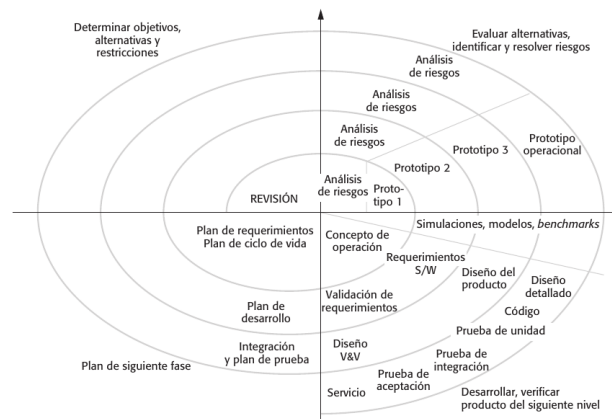


Figura 2.11 Modelo en espiral de Boehm

Cada ciclo en la espiral se divide en cuatro sectores:

Figura 2.11, Sommerville, p. 49; figura atribuida en el libro a IEEE, 1988. Seguí una vuelta completa por sus cuatro sectores. Lo distintivo es evaluar explícitamente riesgos en cada ciclo.

RUP: separar fases de actividades

RUP integra prácticas iterativas y elementos de distintos modelos. Tiene tres perspectivas: **dinámica** (fases en el tiempo), **estática** (flujos de trabajo) y **práctica** (buenas prácticas).

- **Concepción:** caso de negocio, alcance e interacciones externas; decidir si vale la pena.
- **Elaboración:** comprender el dominio, definir arquitectura, riesgos y plan.
- **Construcción:** diseñar, programar, integrar y probar el sistema.
- **Transición:** llevarlo al entorno real de usuarios y ponerlo en operación.

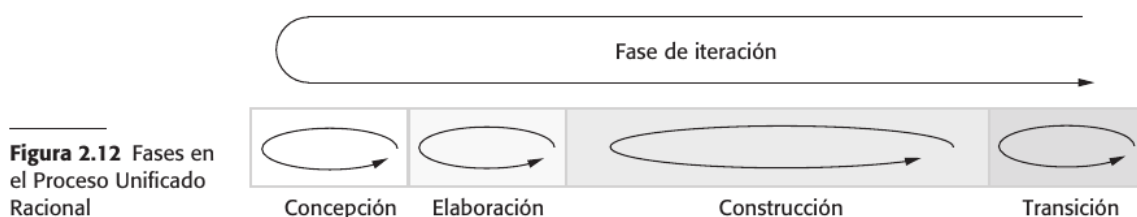


Figura 2.12 Fases en el Proceso Unificado Racional

Figura 2.12, Sommerville, p. 51. Puede haber iteraciones dentro de cada fase y nuevos ciclos completos. Una fase no equivale a una única actividad técnica.

Flujos centrales: negocio, requisitos, análisis/diseño, implementación, pruebas y despliegue. **De apoyo en RUP:** configuración/cambio, gestión del proyecto y entorno. Esta clasificación específica no debe confundirse con la terna genérica de apoyo del formulario.

Seis prácticas: desarrollar iterativamente, gestionar requisitos, usar componentes, modelar visualmente, verificar calidad y controlar cambios. UML es un lenguaje de modelado; RUP es un proceso.

12. SCM y elementos de configuración

Gestión de configuración del software

SCM significa Software Configuration Management. Identifica y controla el software y sus componentes durante construcción, mantenimiento y uso, mediante políticas, procesos y herramientas. Permite saber qué cambió, quién lo cambió y qué versiones constituyen un producto.

Elemento de configuración, ECS o SCI: artefacto identificado y puesto bajo control. Ejemplos: código fuente, documento de requisitos, diseño, pruebas, datos de prueba, scripts y archivos de configuración. Una impresora o un servidor físico no es la respuesta esperada frente a «código fuente» en el formulario.

Versión: estado identificable de un elemento. **Configuración:** combinación de versiones que forma un sistema. **Línea base:** conjunto controlado de versiones que permite reconstruir un estado del sistema; para cambiarlo se establece una nueva configuración controlada. **Release:** versión entregada para uso. No todo commit es una release.

Actividades y relación con Git

- **Gestión de cambios:** registrar solicitudes, analizar impacto y costo y decidir aceptación y momento de implementación.
- **Control de versiones:** registrar y recuperar versiones y coordinar trabajo concurrente. Es el foco principal de Git.
- **Construcción:** ensamblar, compilar y vincular versiones adecuadas de código, datos y bibliotecas.
- **Gestión de entregas:** preparar y registrar qué versiones se distribuyen a usuarios.

Estas cuatro actividades corresponden a la descripción de Sommerville, capítulo 25. Se incluyen como **complemento acotado** porque las capturas preguntan por SCM; ese capítulo no forma parte de los cinco capítulos que pediste resumir.

Diferencia de terminología en el formulario. Su pregunta sobre «tres actividades principales» ofrece como alternativa b «Gestión de cambios, Control de versiones y Gestión de la configuración». Es la opción que mejor encaja entre las disponibles, pero es circular porque SCM ya significa gestión de configuración. No es la lista de cuatro actividades del libro. La foto marca tu elección c como incorrecta, pero no muestra una clave oficial de b: aquí se señala como respuesta probable por descarte, no como confirmación del docente.

SCM no equivale a Git. Un repositorio puede conservar versiones y ramas, pero por sí solo no decide prioridades de cambios ni certifica calidad ni ejecuta toda la gestión del proyecto.

Para recordar: gestión de proyectos organiza trabajo; aseguramiento de calidad organiza cómo procurar y evaluar calidad; SCM controla artefactos, cambios y versiones.

Fuente: formularios; complemento de Sommerville, pp. 682-684.

13. Git: control de versiones distribuido

Pro Git, capítulo 1

Un sistema de control de versiones registra cambios para recuperar estados anteriores, comparar versiones e investigar quién cambió qué. Puede versionar archivos que no son código, aunque las diferencias de texto suelen ser más fáciles de revisar que las de archivos binarios.

Local: historial en una máquina. **Centralizado:** historial principal en un servidor; su indisponibilidad limita operaciones de colaboración y registro central. **Distribuido:** cada clon normal dispone de un repositorio e historial local, además de poder intercambiar cambios con otros repositorios.

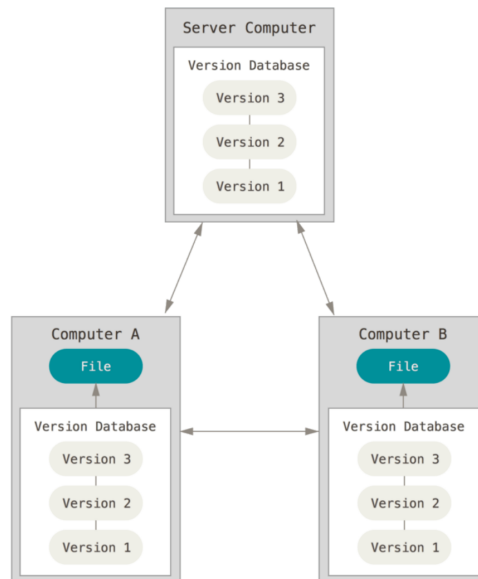


Figura 3, Pro Git, p. 12. Cada computadora tiene su propia base de versiones. El servidor puede coordinar al equipo, pero no es imprescindible para crear un commit local.

Git nació en 2005, impulsado por Linus Torvalds y el desarrollo del kernel de Linux tras el conflicto con BitKeeper. Objetivos: velocidad, diseño sencillo, desarrollo no lineal, distribución y manejo de proyectos grandes.

Trabajo offline: editar, preparar, confirmar, consultar historial disponible, crear ramas y fusionar son operaciones locales. Para intercambiar datos con un servidor de Internet se necesita conectividad. «Distribuido» no impide adoptar un servidor central por convención del equipo.

Git y GitHub son distintos. Git es el sistema de versiones; GitHub es un servicio que puede alojar repositorios y agregar colaboración. Un repositorio Git puede existir sin GitHub ni remoto.

Integridad. El libro explica identificadores basados en SHA-1 de 40 caracteres hexadecimales y comprobaciones de contenido. No son números consecutivos, contraseñas ni una firma del autor. Esa descripción corresponde al formato explicado en la edición.

Fuente: Pro Git, pp. 9-16. La explicación del clon presupone un clon normal, no modalidades parciales o superficiales.

14. Instantáneas y las tres áreas de Git

Git modela el historial como **instantáneas** del proyecto. Si un archivo no cambió, puede reutilizar la referencia a su contenido. No significa que cada commit duplique físicamente todos los archivos.

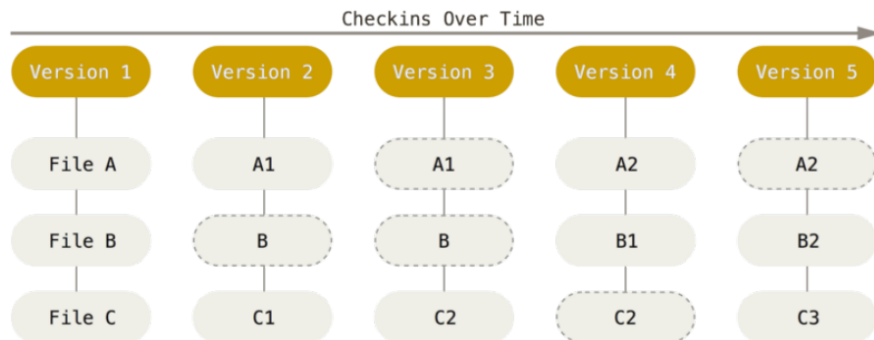


Figura 5, Pro Git, p. 14. Cada columna es una versión del proyecto. Los archivos sin cambios reutilizan contenido; los modificados incorporan una nueva versión.

Directorio de trabajo: archivos que ves y editas. **Staging area o índice:** contenido preparado para la próxima confirmación. **Repositorio local, directorio .git:** objetos e información del historial. El índice está normalmente dentro de .git, pero cumple una función distinta del historial confirmado.

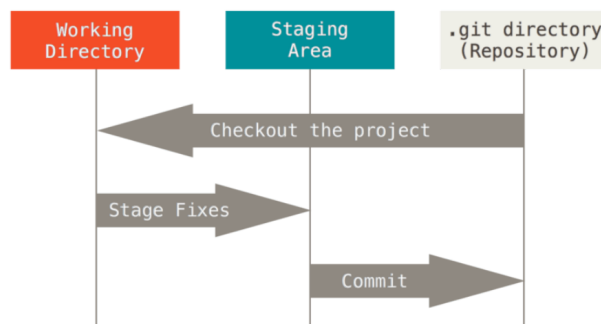


Figura 6, Pro Git, p. 16. Stage files corresponde a preparar; commit registra la instantánea del índice. Checkout actualiza la copia de trabajo a partir de una versión.

Estados fundamentales: modified significa modificado; staged, preparado; committed, confirmado. Además, hay archivos **untracked** que aún no están siendo seguidos y archivos tracked sin modificaciones.

Regla decisiva: un commit normal guarda el contenido que preparaste, no necesariamente lo último que ves en el editor. Si ejecutás add y luego editás otra vez, necesitás otro add para preparar esa nueva edición.

Ejemplo mental: historial=A; editás a B; hacés add; editás a C. Ahora historial=A, índice=B y directorio de trabajo=C. Un commit normal registra B; C queda como cambio sin preparar.

15. Configuración, ayuda e inicio

Identidad antes de confirmar

La identidad usada para commits debe estar disponible. La forma habitual de configurarla es:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
git config --list
git config user.name
```

--global aplica al usuario; --system, a la instalación/sistema; la configuración local de .git/config, al repositorio. Entre estos tres niveles, lo local prevalece sobre global y global sobre sistema. Sin --global, establecer una variable dentro de un repositorio normalmente la configura allí.

Nombre y correo identifican al autor; no autentican ante un servidor. No hace falta crear .gitignore, establecer una URL remota ni elegir manualmente un editor para realizar todo commit. Con -m escribís el mensaje en el comando.

Editor y ayuda: git config --global core.editor "." configura el editor apropiado; git help commit o git commit --help consultan ayuda. La disponibilidad local del manual depende de la instalación. git --version permite ver la versión instalada.

Crear o clonar

```
git init
git clone URL carpeta
git status
```

init: inicializa el repositorio en el directorio indicado o actual y crea su estructura .git. No crea automáticamente el primer commit ni sube el proyecto. **clone:** copia un repositorio existente; normalmente configura origin, obtiene su historial y deja una rama local activa siguiendo la rama predeterminada remota.

Línea de comandos. Da acceso a las operaciones de Git; las interfaces gráficas ofrecen otra forma de invocarlas y pueden exponer solo un subconjunto. No realizan otro tipo de versionado. Git está disponible en los principales sistemas operativos, como Windows, Linux y macOS. La opción del formulario que dice «todos los sistemas operativos» debe entenderse con ese alcance, no literalmente todo sistema existente.

Instalación, contenido conceptual del capítulo. Se puede instalar mediante paquetes, instaladores o compilación del código fuente. Los nombres concretos de instaladores del libro son históricos; para el escrito importa distinguir instalación, configuración de identidad y creación del repositorio.

Fuente: Pro Git, pp. 17-25.

16. Seguimiento, add y commit

Ciclo de vida de los archivos

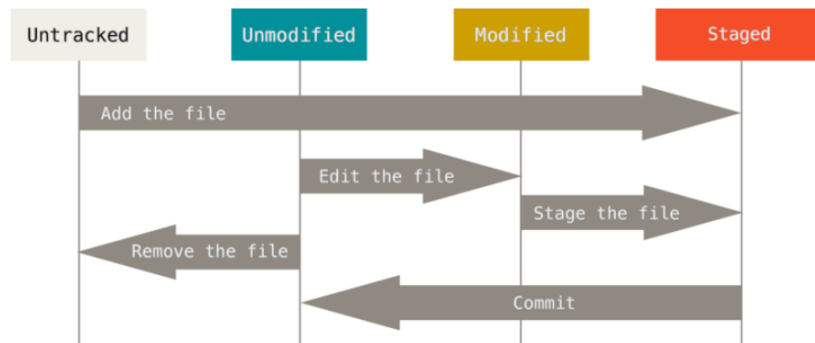


Figura 8, *Pro Git*, p. 26. Un archivo nuevo comienza untracked; add lo incorpora al seguimiento y lo prepara. Editar un archivo seguido lo modifica; preparar y confirmar son pasos diferentes.

```
git status
git add archivo.txt
git add .
git commit -m "Explicación del cambio"
git commit -am "Actualizar archivos seguidos"
```

status informa estados. **add** incorpora al índice el contenido actual de la ruta elegida. Sirve para archivos nuevos, cambios y, según la ruta, eliminaciones. **add .** actúa desde el directorio actual hacia abajo: desde la raíz cubre el proyecto; desde una subcarpeta no equivale a seleccionar cualquier ruta del repositorio. Los archivos ignorados no se agregan normalmente.

commit -m registra localmente el índice con un mensaje. No publica. **commit -a** prepara automáticamente modificaciones y eliminaciones de archivos ya seguidos antes de confirmar; no incluye por sí solo archivos nuevos untracked. Si había contenido preparado, también forma parte del commit normal de ese flujo.

Leer el estado abreviado

```
?? nuevo.txt
M pendiente.txt
M preparado.txt
MM dos-versiones.txt
A incorporado.txt
```

La primera columna compara índice con HEAD; la segunda, trabajo con índice. MM puede significar que se preparó una modificación y después se volvió a editar. A indica una incorporación preparada. ?? es sin seguimiento.

Trampa: guardar en el editor no es add; add no es commit; commit no es push. «Working tree clean» tampoco significa que no haya commits locales pendientes de publicar: describe las áreas de trabajo e índice respecto del estado confirmado.

Fuente: *Pro Git*, pp. 25-35.

17. Comparar cambios y consultar historia

status, diff, log y show

- `git status`: qué archivos están en cada estado.
- `git diff`: diferencias del trabajo respecto del índice; cambios sin preparar de archivos seguidos.
- `git diff --staged`: diferencias del índice respecto de HEAD; qué cambiaría el próximo commit. `--cached` es sinónimo aquí.
- `git log`: historial de commits alcanzables desde la referencia consultada, normalmente la rama actual; incluye identificador, autor, fecha y mensaje.
- `git show IDENTIFICADOR`: detalle de un commit u otro objeto; para un commit usual incluye información y cambios.

Si `add` preparó todo, `diff` puede no mostrar nada aunque aún falte hacer commit. Usá `diff --staged` para revisar lo preparado. Un archivo untracked no aparece como un parche normal en `git diff` solo por existir.

```
git log --oneline
git log -p -2
git log --stat
git log --oneline --decorate --graph --all
git log --author="Ana"
git log --since="2026-09-01"
git log --grep="error"
git log -- archivo.txt
git log -S"nombreFuncion"
```

oneline: resumen de una línea. **-p**: parches. **-2**: limita a dos commits. **stat**: estadísticas por archivo. **decorate**: nombres de referencias. **graph**: dibujo de ramificaciones. **all**: considera todas las referencias seleccionadas por esa opción. **author**: filtra autores. **since/until**: fechas. **grep**: mensajes. **-S**: cambios en el número de apariciones de una cadena. `--` ruta separa rutas de opciones/revisiones.

El autor escribió el trabajo; el **committer** lo incorporó al historial. Pueden ser personas diferentes, por ejemplo al aplicar un parche ajeno.

Lectura de un diff: las líneas con `-` se eliminan y las de `+` se agregan; las cabeceras señalan archivos y posiciones. No confundas las marcas del parche con comandos para ejecutar.

Ejemplo del formulario: «historial con autor, fecha y mensaje» apunta a `git log`. `status` muestra el presente; `diff` compara contenidos; `show` suele inspeccionar un objeto particular.

Fuente: *Pro Git*, pp. 31-35 y 38-45.

18. Ignorar, mover y deshacer

Archivos ignorados

.gitignore declara patrones de archivos que no se desea incorporar normalmente al seguimiento. Se usa para compilados, temporales y archivos generados. Puede versionarse para compartir las reglas.

```
# Comentario
*.o
build/
/temporal.txt
!importante.o
doc/**/*.*tmp
```

* representa caracteres; ?, un carácter; [abc], alternativas; / inicial ancla al directorio del .gitignore; / final indica directorio; ! niega una regla, con las limitaciones de directorios excluidos; ** permite niveles de directorios. Las líneas vacías y comentarios se omiten.

No deja de seguir un archivo ya versionado ni borra su historial. Para conservarlo en disco y preparar que deje de estar versionado se usa `git rm --cached archivo`, luego se confirma y se agregan las reglas apropiadas.

Operaciones con archivos

`git rm archivo` elimina del trabajo y prepara la eliminación del seguimiento. `git mv viejo nuevo` mueve/renombra y prepara el cambio. El commit posterior registra esas operaciones. Git puede detectar renombramientos por similitud; no depende exclusivamente de usar mv.

Tres acciones diferentes al deshacer

```
git reset HEAD archivo.txt
git checkout -- archivo.txt
git commit --amend
```

reset HEAD archivo: saca del índice los cambios respecto de HEAD, conservando el archivo de trabajo. Este ejemplo presupone que existe un commit HEAD. **checkout -- archivo:** restaura en el trabajo el contenido del índice; si el índice coincide con HEAD, vuelve a la última versión confirmada. Descarta modificaciones sin preparar de esa ruta.

amend: reemplaza el último commit por uno nuevo que incorpora el contenido preparado y permite corregir el mensaje. Cambia la identidad del commit: no es agregar una segunda confirmación normal. Usalo para trabajo local aún no compartido, en el contexto del libro.

Distinción esencial: quitar del staging no borra las ediciones; descartar ediciones sí modifica el archivo. Los cambios nunca confirmados pueden no recuperarse. Se explica ese efecto porque es parte de saber elegir el comando correcto.

Fuente: Pro Git, pp. 30-31, 36-38 y 45-48. Se usa la sintaxis del libro.

19. Repositorios remotos

Un remoto es una referencia con nombre a otro repositorio. Puede estar en Internet, una red o una ruta local. `origin` es un nombre convencional que clone suele crear, no una palabra mágica ni una rama.

```
git remote -v
git remote add origin URL
git remote show origin
git remote rename origin central
git remote rm central
```

Listar con `-v` muestra direcciones; `add` registra una dirección, no crea el repositorio en el servidor ni sube cambios. Renombrar o eliminar un remoto cambia la configuración local; `remote rm` no elimina el servidor.

fetch, pull y push

fetch origin: obtiene objetos y actualiza referencias de seguimiento remoto; no fusiona automáticamente esos cambios en la rama local activa.

pull: obtiene e integra en la rama actual. El flujo básico del libro es `fetch` seguido de `merge`; también puede configurarse para `rebase`. No es correcto afirmar que `pull` siempre deja el directorio de trabajo intacto.

push origin master: envía los commits y actualiza la rama correspondiente del remoto si está permitido. No envía cambios que solo estén en el editor o en staging.

```
git fetch origin
git checkout master
git merge origin/master
git push origin master
```

Este flujo separa la descarga de la integración. Si surge un conflicto, debe resolverse antes de completar la integración.

Rechazo de push. Una causa común es que el remoto tenga commits que tu rama no contiene. Obtén e integrá esos cambios antes de volver a publicar. No es cierto que cualquier nuevo commit remoto obligue a rechazar: el problema es que la actualización solicitada no avance de forma compatible, además de posibles permisos o políticas del servidor.

Autenticación y autoría. Poder leer no implica poder escribir. El nombre y correo configurados no conceden permisos de publicación.

Escenario de examen: hiciste dos commits offline. Siguen siendo commits válidos en el repositorio local aunque nunca hayas configurado un remoto. Cuando quieras compartirlos, configurarás el destino y publicarás con conectividad y permisos adecuados.

Fuente: Pro Git, pp. 48-53 y 80-90.

20. Etiquetas y alias

Tags: marcar puntos del historial

Una etiqueta identifica un punto concreto, por ejemplo una versión entregada. No avanza automáticamente cuando se hacen nuevos commits, a diferencia de una rama activa.

Ligera: referencia simple a un commit. **Anotada:** objeto con información del etiquetador, fecha y mensaje; puede firmarse. La firma es distinta del identificador hash y no aparece automáticamente por crear cualquier tag.

```
git tag
git tag -l "v1.*"
git tag v1.0
git tag -a v1.1 -m "Primera versión completa"
git tag -a v0.9 IDENTIFICADOR -m "Versión anterior"
git show v1.1
git push origin v1.1
git push origin --tags
```

Podés etiquetar el commit actual o uno anterior. En el flujo estándar del libro, publicar una rama no publica automáticamente todas las etiquetas; se envían explícitamente.

Trabajar a partir de una etiqueta:

```
git checkout -b correccion-v1 v1.0
```

Crea una rama desde el punto etiquetado y cambia a ella. Los nuevos commits avanzan esa rama, no mueven la etiqueta. Hacer checkout directo de un commit o tag puede dejar HEAD separado; crear una rama facilita conservar nuevo trabajo con un nombre.

Alias: abreviaturas configuradas

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.last "log -1 HEAD"
git st
git last
```

st no es un comando universal que debas asumir en cualquier computadora: funciona después de configurar ese alias. Los alias se expanden a comandos; el libro también muestra ! para invocar comandos externos.

Conceptos que suelen cruzarse: un commit registra un estado; una rama permite desarrollar una línea; un tag marca un punto; una release es una entrega para uso. Pueden relacionarse, pero no son sinónimos.

Fuente: Pro Git, pp. 53-58. Aclaración técnica: el texto antiguo sobre no poder hacer checkout de un tag debe interpretarse como que el tag no es una rama móvil, no como prohibición literal del comando.

21. Ramas y HEAD

Pro Git, capítulo 3

Una rama es una referencia móvil a un commit. Crear una rama no duplica todos los archivos ni crea otro repositorio. HEAD normalmente apunta a la rama local actual; esa rama avanza cuando se hace un nuevo commit.



Figura 13, Pro Git, p. 62. HEAD apunta a master y master a un commit. testing puede apuntar al mismo commit sin estar activa. Crear testing no cambia HEAD por sí solo.

```

git branch nueva
git checkout nueva
git checkout -b otra
git branch
git branch -v
git branch --merged
git branch --no-merged
git branch -d terminada
  
```

branch nueva: crea. **checkout nueva:** cambia. **checkout -b otra:** crea y cambia. **branch** lista y marca con asterisco la activa. **-v** agrega último commit. **--merged/--no-merged** evalúan integración respecto de la referencia usada, normalmente HEAD. **-d** elimina una rama con comprobaciones; no elimina la rama activa. **-D** fuerza y omite protección de integración, por lo que no equivale a **-d**.

Cambiar de rama actualiza HEAD, índice y archivos de trabajo cuando puede hacerlo sin sobrescribir cambios incompatibles. No exige siempre que no haya ninguna edición: puede conservar cambios compatibles y rechazar el cambio cuando perdería trabajo.

Ramas de larga duración: representan niveles de estabilidad, como producción y desarrollo. **Ramas de tema:** se crean para una funcionalidad, corrección o experimento y se integran al terminar. Son convenciones de trabajo, no reglas impuestas por Git.

HEAD separado: apunta directamente a un commit en vez de a una rama. Es útil para inspeccionar; si se quiere continuar trabajo con referencia permanente, se crea una rama.

Objetos básicos del capítulo: blob almacena contenido; tree relaciona nombres y contenidos/subárboles; commit referencia el árbol, sus padres y metadatos. El primer commit no tiene padre; uno ordinario suele tener uno; una fusión típica tiene dos.

Fuente: Pro Git, pp. 59-65 y 75-80.

22. Integrar ramas y resolver conflictos

merge modifica la rama actual

Para integrar tema en master, primero activás master y después fusionás tema:

```
git checkout master
git merge tema
```

Fast-forward: si master es ancestro de tema, puede adelantarse el puntero sin crear un commit de fusión.

Fusión de tres vías: si ambas líneas divergieron, se comparan sus extremos y ancestro común; una fusión normal exitosa crea un commit con ambos padres.

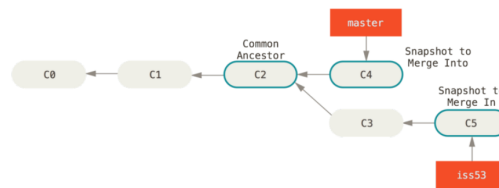


Figura 24, Pro Git, p. 71. El ancestro común y los dos extremos aportan la información para la fusión. No se elige simplemente el archivo cuya fecha sea más reciente.

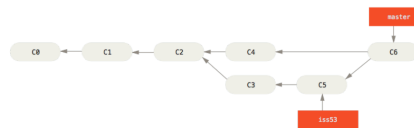


Figura 25, Pro Git, p. 72. El nuevo commit conecta ambas historias. La rama integrada no desaparece por hacer merge; puede borrarse después si corresponde.

Conflictos

Ocurren cuando Git no puede conciliar cambios automáticamente, por ejemplo modificaciones incompatibles en las mismas líneas. Editar el mismo archivo en dos ramas no implica siempre conflicto.

```
<<<<<< HEAD
Versión de la rama actual
=====
Versión de la rama que se integra
>>>>>> tema
```

Se inspecciona `git status`, se editan los archivos para dejar el resultado correcto y se quitan las marcas. Luego:

```
git add archivo-resuelto.txt
git commit -m "Integrar tema y resolver diferencias"
```

add marca la resolución; no decide si el contenido es correcto. Hay que comprobarlo. `git mergetool` permite apoyarse en una herramienta de fusión.

Modelo commit-before-merge del formulario: la alternativa b es la compatible entre las ofrecidas: Git permite confirmar trabajo local y después integrarlo. No significa que todo merge deba estar precedido inmediatamente por un nuevo commit ni que toda integración genere uno.

23. Ramas remotas y seguimiento

Hay que distinguir **master local**, **master en el servidor** y **origin/master en tu repositorio**, que registra el último estado remoto conocido. Esta última no es una consulta en vivo al servidor.

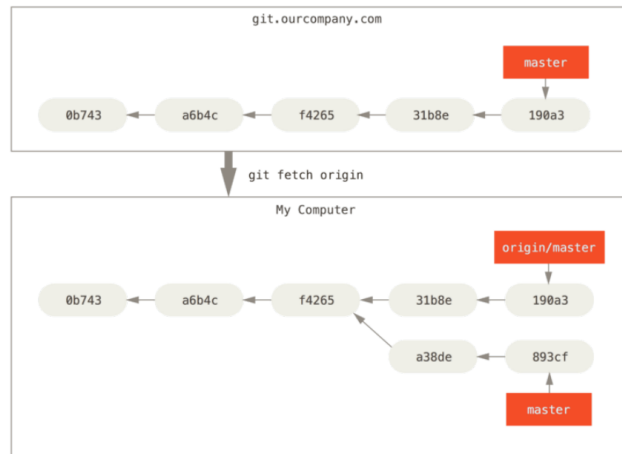


Figura 32, *Pro Git*, p. 84. *fetch* actualiza *origin/master* según el servidor; tu *master* conserva su línea de trabajo hasta que decidas integrar.

Rama de seguimiento: rama local con upstream configurado. Esa relación facilita saber de dónde traer cambios. Una referencia *origin/rama* y una rama local que sigue *origin/rama* cumplen papeles diferentes.

```
git fetch origin
git checkout -b tema origin/tema
git checkout --track origin/otra
git branch -u origin/tema
git branch -vv
git push -u origin tema
git push origin tema:nombre-remoto
git push origin --delete tema
```

`checkout -b` crea una rama local desde el estado remoto conocido. `--track` configura seguimiento. `branch -u` cambia el upstream de la rama actual. `push -u` publica y configura seguimiento. `tema:nombre-remoto` relaciona fuente local con destino remoto. `push --delete` elimina la rama del servidor; `branch -d` elimina una rama local.

ahead 2, behind 1: hay dos commits solo en tu rama y uno solo en la referencia de seguimiento. La información depende del último intercambio; consultá `fetch` para actualizar tu conocimiento antes de sacar conclusiones sobre el servidor.

Al clonar, se obtienen referencias de las ramas remotas, pero no se crea una rama local activa por cada una. Crear una nueva rama local tampoco la publica automáticamente.

Fuente: *Pro Git*, pp. 80-90.

24. Rebase y comparación con merge

Rebase reaplica cambios de commits sobre otra base y produce nuevos commits. Puede dejar una historia lineal, pero cambia los identificadores de los commits reaplicados. No es un simple cambio de nombre de la rama.

```
git checkout experimento
git rebase master
git checkout master
git merge experimento
```

Primero se reaplica el trabajo de experimento sobre master. Después master puede avanzar hasta el resultado. Rebase no actualiza automáticamente cualquier otra rama relacionada.

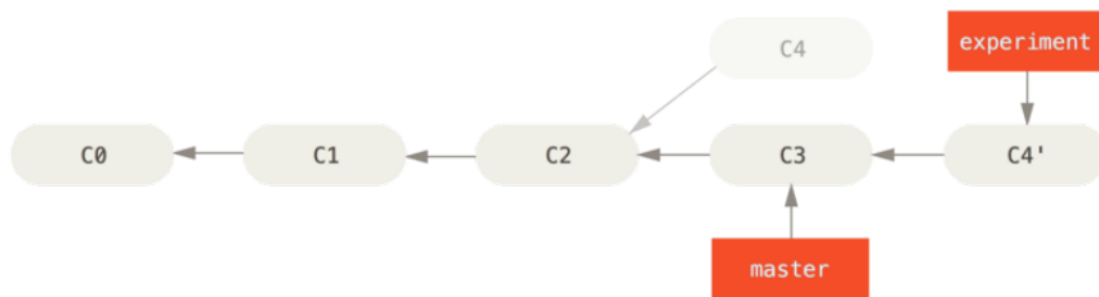


Figura 37, *Pro Git*, p. 91. C4 se reaplica como C4' sobre la nueva base. El contenido integrado puede coincidir con el de un merge, pero la historia y los identificadores son diferentes.

Merge conserva los commits existentes y conecta historias cuando hace falta. **Rebase** reconstruye la secuencia reaplicada. No hay una opción universalmente superior: importa qué historia quiere conservar el equipo y si el trabajo ya fue compartido.

Regla práctica del libro: no reorganizar commits publicados sobre los que otras personas pudieron basar trabajo. Hacerlo deja historias viejas y nuevas que deben reconciliarse; puede producir duplicación aparente y confusión.

Variantes del capítulo

`git rebase master tema` selecciona tema y la reorganiza sobre master. `git rebase --onto master servidor cliente` permite trasladar el trabajo propio de cliente que no pertenece a servidor a una base nueva. En el ejemplo del libro, sirve para incorporar el lado cliente sin llevar todavía los cambios del servidor.

Si hay conflictos en rebase: resolver el contenido, prepararlo con `add` y continuar con `git rebase --continue`. `git rebase --abort` cancela la operación y vuelve al estado anterior al rebase. Esta secuencia completa la práctica operativa del concepto; no se resuelve exactamente con el mismo cierre manual que un merge normal.

No confundas: `fetch` descarga; `merge` integra historias; `rebase` reaplica commits; `pull` obtiene e integra según configuración; `push` publica. Todos salvo el intercambio con el remoto pueden realizarse con los datos locales necesarios.

Fuente: *Pro Git*, pp. 90-99.

25. Formulario A: respuestas explicadas

Capturas terminadas en 150233 y 150240

Se conservan las letras según tus imágenes. Este es el formulario que comienza preguntando por las características del software.

- 1. a. Abstracto e intangible; puede alcanzar gran complejidad.** No se caracteriza por desgaste físico como el hardware ni por límites de materiales.
- 2. a. Mostrar que cumple especificación y expectativas del cliente.** Es el propósito de validación/V&V; no basta corregir sintaxis o producir documentación.
- 3. d. Demostrar conceptos, probar opciones y refinar requisitos con usuarios.** Un prototipo no es obligatoriamente el sistema final ni sustituye toda documentación.
- 4. b. Incorporar cambios en requisitos a un costo relativamente bajo.** Es una ventaja del desarrollo incremental. Requisitos estables desde el inicio favorecen cascada, y la estructura puede necesitar refactorización.
- 5. d. Programas con documentación, bibliotecas, soporte y configuración.** El producto software es más amplio que código o ejecutables.
- 6. c. Disciplina de ingeniería que abarca todos los aspectos de producción del software.** Incluye desde especificación hasta mantenimiento. La opción sobre hardware y procesos corresponde al alcance mayor de ingeniería de sistemas.
- 7. c. git init inicializa un repositorio en la ubicación actual.** No crea por sí solo un commit, una rama llamada init ni un repositorio en la nube.
- 8. b. Código fuente.** Es un ejemplo directo de elemento de configuración de software; la ubicación de un servidor no lo convierte en ese artefacto.
- 9. b. Modelo commit-before-merge.** Es la alternativa compatible con el flujo de confirmar localmente antes de integrar. Las otras limitan falsamente Git a un único repositorio, Internet para confirmar o ramas solo centralizadas. No tomes esa etiqueta como regla absoluta para cada merge.
- 10. c. Distribuido.** Git permite repositorios e historial locales y colaboración con varios remotos.
- 11. b, probable por las opciones; redacción problemática.** «Gestión de cambios, Control de versiones y Gestión de la configuración». La captura no muestra una corrección oficial de esa opción. Sommerville enumera cambio, versiones, construcción y entregas; ver la aclaración en la sección de SCM.
- 12. a. push envía commits del repositorio local al remoto.** La opción «obtener y fusionar» describe pull. push no confirma cambios del directorio de trabajo.

26. Formulario B: respuestas explicadas

Capturas terminadas en 150259 y 150308

Es el formulario cuyo primer enunciado queda recortado. Se ven las opciones y la c señalada como correcta; la identificación del tema SCM se deduce de ese texto.

- 1. c. Identificación y control del software y sus componentes durante construcción, mantenimiento y uso.** Corresponde a gestión de configuración. Planificar actividades es gestión de proyectos; asegurar calidad es otro objetivo.
- 2. c. «Arte de desarrollar productos ...» NO es la definición adecuada entre esas alternativas.** Las otras destacan disciplina, sistematicidad y aplicación de conocimiento bajo restricciones. La ingeniería puede incluir creatividad, pero no se define solo como arte.
- 3. b. Crisis del software.** La referencia es la conferencia de 1968 y los problemas de costo, plazos y confiabilidad.
- 4. c. Especificación, desarrollo, validación y evolución.** Aprendí la lista y el propósito de cada actividad; las otras alternativas mezclan tareas particulares.
- 5. d. Gestión de proyectos, gestión de configuración y aseguramiento de calidad.** Es la terna de apoyo ofrecida. No confundir con las actividades fundamentales ni con los flujos de apoyo específicos de RUP.
- 6. c. Entender y definir servicios requeridos y restricciones de operación.** Especificación/ingeniería de requisitos. Comprobar expectativas corresponde a validación.
- 7. d. git log.** Muestra historial de commits con autor, fecha y mensaje. show inspecciona un objeto concreto; diff compara cambios; status informa estados.
- 8. c. Nombre y email del desarrollador.** La identidad debe estar disponible para el commit. No es necesario configurar previamente un repositorio remoto.
- 9. a. git add . prepara los cambios en staging.** La opción es la correcta, pero está resumida: también puede incorporar archivos nuevos y preparar eliminaciones dentro del alcance de la ruta. No crea el commit.
- 10. c. Archivo nuevo en el trabajo aún no incorporado al índice ni al último commit.** Eso describe untracked. La ausencia del área de preparación por sí sola no convierte en untracked a un archivo ya seguido.
- 11. c, con precisión de lenguaje.** La CLI está disponible en los principales sistemas operativos. «Todos» es una simplificación del formulario. No usa operaciones de versionado distintas de una GUI ni requiere otra herramienta de versionado adicional.
- 12. a. Versionar localmente y trabajar de manera autónoma offline.** Crear commits y ramas no exige servidor. La conectividad es necesaria cuando el remoto que se quiere contactar está en una red.

27. Preguntas: Software y fundamentos

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

01. ¿Por qué software no significa solamente código?

Porque incluye programas, documentación y datos de configuración necesarios para usarlo y mantenerlo.

02. ¿Qué vuelve complejo al software pese a ser intangible?

Su flexibilidad permite acumular funciones, interacciones y cambios sin las mismas restricciones físicas del hardware.

03. ¿Qué diferencia la producción profesional de un programa personal?

Debe atender a otros usuarios y mantenedores, gestionar calidad y restricciones y conservar información que permita sostener el producto.

04. ¿Quién controla normalmente los requisitos de un producto genérico?

La organización que lo desarrolla; en uno a medida, el cliente suele controlar la especificación.

05. ¿La ingeniería de software termina cuando compila el programa?

No. Incluye pruebas, documentación, entrega, mantenimiento y gestión, además de otras tareas previas al código.

06. ¿Cuál es la diferencia entre ingeniería de sistemas y de software?

La primera incluye hardware, software, procesos y su integración; la segunda se concentra en la producción del software.

07. ¿Qué problemas motivaron la crisis del software?

Retrasos, sobre costos, baja confiabilidad y sistemas que no satisfacían necesidades al aumentar la escala.

08. ¿Cuáles son los cuatro atributos esenciales del buen software?

Mantenibilidad, confiabilidad y seguridad, eficiencia y aceptabilidad.

09. ¿Funcionar correctamente garantiza aceptabilidad?

No: también debe ser comprensible, utilizable y compatible con el contexto del usuario.

10. ¿La proporción 60% desarrollo y 40% pruebas vale para todo proyecto?

No; es una orientación del texto. Tipo de sistema, riesgos y vida útil cambian los costos.

28. Preguntas: Diversidad, ética y casos

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

11. ¿Qué tipo de sistema procesa una nómina mensual?

Procesamiento por lotes: transforma un conjunto de entradas en resultados de forma agrupada.

12. ¿Qué tipo de sistema controla una bomba o frenos?

Control embebido, que interactúa con hardware; puede ser crítico para la seguridad.

13. ¿Qué distingue una simulación de una adquisición de datos?

La simulación modela un proceso; la adquisición recoge mediciones del entorno mediante sensores.

14. ¿Hay un proceso óptimo para cualquier sistema?

No. Se elige según requisitos, riesgos, tipo de aplicación, personas y restricciones.

15. ¿Cuáles son los retos generales del capítulo 1?

Heterogeneidad; cambio empresarial y social y rapidez de entrega; seguridad y confianza.

16. ¿Cómo cambió la Web el desarrollo de software?

Impulsó distribución, servicios, reutilización, despliegue centralizado y evolución incremental.

17. ¿Sin acuerdo escrito desaparece la confidencialidad profesional?

No. El libro presenta responsabilidades profesionales más amplias que los acuerdos formales.

18. ¿Qué implica actuar con competencia?

Describir honestamente las capacidades y no asumir deliberadamente trabajos para los que no se está preparado.

19. ¿Por qué se usan tres estudios de caso distintos?

Para mostrar que las técnicas y prioridades varían entre sistemas embebidos, de información y de adquisición de datos.

20. ¿Qué tensión ilustra el sistema de pacientes?

Privacidad frente a disponibilidad: más copias ayudan a acceder a datos, pero complican su protección.

29. Preguntas: Procesos y modelos

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

21. ¿Proceso y modelo de proceso son sinónimos?

No. El proceso reúne actividades reales; el modelo es una representación parcial y simplificada.

22. ¿Qué cuatro actividades deben aparecer en cualquier proceso?

Especificación, desarrollo, validación y evolución, aunque se organicen de formas diferentes.

23. ¿Qué diferencia un rol de un producto del proceso?

El rol define responsabilidades; el producto es un resultado, como un modelo o documento.

24. ¿Qué son precondiciones y postcondiciones?

Condiciones que deben cumplirse antes y después de una actividad, respectivamente.

25. ¿Qué distingue planificación anticipada de planificación ágil?

La primera organiza por adelantado; la segunda planifica incrementalmente y facilita adaptación. Ambas planifican.

26. ¿Cuáles son las fases de cascada?

Requisitos; diseño; implementación y prueba de unidades; integración y prueba del sistema; operación y mantenimiento.

27. ¿Cascada prohíbe volver a una fase anterior?

No. Hay retroalimentación, pero el costo de revisar decisiones y documentos puede ser alto.

28. ¿Qué tres ventajas principales ofrece desarrollo incremental?

Adaptar cambios a menor costo, obtener retroalimentación más fácilmente y entregar valor temprano.

29. ¿Qué dos dificultades destaca el libro para incremental?

Menor visibilidad documental del proceso y deterioro estructural si no se invierte en refactorización.

30. ¿Incremental y ágil son equivalentes?

No. Los incrementos pueden planificarse anticipadamente o mediante un enfoque ágil.

30. Preguntas: Requisitos, pruebas y cambio

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

31. ¿Por qué se modifican requisitos en un proceso de reutilización?

Porque los componentes encontrados pueden no ajustarse exactamente y hay que negociar compatibilidad o buscar alternativas.

32. ¿Reutilizar elimina la necesidad de integrar y validar?

No. Las piezas pueden funcionar individualmente y fallar al interactuar o no satisfacer la necesidad completa.

33. ¿Qué evalúa un estudio de factibilidad?

Si conviene y puede construirse el sistema con tecnología, presupuesto y restricciones disponibles.

34. ¿Requisitos de usuario y del sistema tienen igual detalle?

No. Los primeros comunican necesidades de alto nivel; los segundos especifican con mayor precisión.

35. ¿Diseño de interfaz significa solamente diseño de pantallas?

No. También define contratos entre componentes, necesarios para integrarlos.

36. ¿Qué distingue verificación de validación?

Verificación evalúa conformidad con la especificación; validación evalúa necesidades reales y utilidad prevista.

37. ¿Qué diferencia prueba de depuración?

La prueba revela defectos; la depuración investiga la causa y la corrige.

38. ¿Para qué sirven pruebas de sistema si las unidades ya funcionan?

Para comprobar interacciones, interfaces y propiedades del conjunto.

39. ¿Qué aporta una prueba de aceptación?

Evidencia con participación/datos del cliente de que el sistema resulta aceptable para el uso previsto.

40. ¿Por qué un prototipo desechable no debería entregarse automáticamente?

Puede omitir atributos críticos, documentación y calidad estructural necesarios en producción.

31. Preguntas: Espiral, RUP y SCM

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

41. ¿Desarrollo incremental implica siempre entrega en producción?

No. Se pueden mostrar incrementos sin desplegarlos; entrega incremental sí implica uso operacional.

42. ¿Cuál es la finalidad de refactorizar?

Mejorar estructura interna sin cambiar el comportamiento observable y facilitar cambios posteriores.

43. ¿Qué hace distintivo al modelo en espiral?

El análisis y la reducción explícita del riesgo en cada ciclo.

44. ¿Cuáles son los cuatro sectores de una vuelta de espiral?

Objetivos; evaluación/reducción de riesgos; desarrollo/validación; planificación del siguiente ciclo.

45. ¿Cuáles son las fases de RUP?

Concepción, elaboración, construcción y transición; no son cuatro actividades técnicas exclusivas.

46. ¿Qué perspectivas ofrece RUP?

Dinámica: fases; estática: flujos de trabajo; práctica: recomendaciones de buenas prácticas.

47. ¿RUP asigna pruebas únicamente a una fase final?

No. Los flujos pueden actuar en distintas fases, con énfasis variable e iteraciones.

48. ¿Qué es un elemento de configuración?

Un artefacto identificado bajo control, como código, un documento, pruebas o un script.

49. ¿En qué difieren línea base y release?

La línea base fija una combinación controlada y reproducible de versiones; la release es una entrega para uso.

50. ¿Git cubre toda la gestión de configuración?

No. Proporciona control de versiones; SCM también gestiona cambios, construcciones y entregas.

32. Preguntas: Fundamentos y configuración de Git

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

51. ¿Qué significa que Git sea distribuido?

Que un clon normal tiene repositorio e historial local y puede intercambiar cambios con otros repositorios.

52. ¿Se puede usar un servidor central en Git?

Sí, como convención de colaboración; no cambia su naturaleza distribuida.

53. ¿Se pueden crear commits, ramas y merges sin Internet?

Sí, si los objetos necesarios están disponibles localmente.

54. ¿Por qué Git no guarda simplemente una lista conceptual de diferencias?

Su modelo es de instantáneas del proyecto, reutilizando contenidos sin cambios; puede usar compresión internamente.

55. ¿Un hash de commit es un número de versión secuencial?

No. Es un identificador derivado del objeto; el formato descrito en el libro usa SHA-1.

56. ¿Qué contiene .git?

La base de objetos y metadatos del repositorio, junto con referencias y configuración, entre otros datos.

57. ¿Qué diferencia el índice del directorio de trabajo?

El índice registra el contenido preparado; el directorio de trabajo contiene lo que se ve y edita.

58. ¿Qué configuración del formulario se requiere para confirmar?

Identidad de nombre y correo disponible; no URL remota ni cuenta en GitHub.

59. ¿Qué prevalece entre config local, global y system?

En esos tres ámbitos, local prevalece sobre global y global sobre system.

60. ¿Qué diferencia init de clone?

init inicializa un repositorio; clone obtiene una copia de uno existente con su información de versiones.

33. Preguntas: Estados, historial y archivos

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

61. ¿Qué significa untracked?

Archivo que no está en el último commit ni en el índice, a diferencia de los archivos ya seguidos.

62. ¿add confirma cambios?

No. Prepara el contenido seleccionado para una confirmación posterior.

63. ¿add . siempre abarca todas las carpetas del repositorio?

No. Su alcance parte del directorio actual; desde la raíz abarca el proyecto.

64. ¿Qué se confirma si se hace add y luego se vuelve a editar?

El contenido preparado antes de la nueva edición, salvo que se ejecute otro add u otro flujo que la prepare.

65. ¿commit -a incorpora archivos nuevos untracked?

No por sí solo; prepara modificaciones y eliminaciones de archivos ya seguidos.

66. ¿Qué compara diff sin opciones?

Directorio de trabajo con índice: cambios sin preparar de archivos seguidos.

67. ¿Qué compara diff --staged?

Índice con HEAD: cambios preparados para el siguiente commit.

68. ¿Cuál es el comando de historial con autor, fecha y mensaje?

git log; para inspeccionar un commit concreto suele usarse git show.

69. ¿Agregar un archivo seguido a .gitignore deja de versionarlo?

No. Hay que retirar su seguimiento y confirmar ese cambio si se desea; el historial anterior permanece.

70. ¿En qué se diferencian rm y rm --cached?

rm elimina del trabajo y del índice; rm --cached lo retira del índice conservando el archivo de trabajo.

34. Preguntas: Deshacer, etiquetas y remotos

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

71. ¿reset HEAD elimina sus ediciones de trabajo?

No. En este uso deshace la preparación; el contenido de trabajo se conserva.

72. ¿checkout -- archivo siempre recupera HEAD?

Restaura desde el índice en este uso; coincide con HEAD cuando el índice contiene esa versión.

73. ¿amend añade un commit normal detrás del último?

No. Lo reemplaza por uno nuevo; puede incorporar contenido preparado y cambiar el mensaje.

74. ¿Una etiqueta avanza automáticamente con cada commit?

No. Una rama activa avanza; un tag permanece en su punto salvo que se modifique explícitamente.

75. ¿Qué diferencia un tag anotado de uno ligero?

El anotado tiene objeto y metadatos propios; el ligero es una referencia simple.

76. ¿Cómo se publican las etiquetas en el flujo del libro?

Con git push origin etiqueta o git push origin --tags, de forma explícita.

77. ¿Qué hace remote add origin URL?

Registra un remoto en la configuración local; no crea un servidor ni publica archivos.

78. ¿Qué diferencia fetch de pull?

fetch obtiene sin integrar en la rama activa; pull obtiene e integra según su configuración.

79. ¿push envía un archivo solamente guardado en el editor?

No. Publica commits y referencias, no ediciones sin confirmar.

80. ¿Por qué puede rechazarse un push después de que un compañero publique?

La rama remota puede contener trabajo no integrado que impida un avance compatible; hay que obtener e integrar antes.

35. Preguntas: Ramas y fusión

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

81. ¿Qué es una rama en Git?

Una referencia móvil a un commit; no una copia independiente de todo el repositorio.

82. ¿Qué hace branch tema con HEAD?

Crea tema, pero no cambia la rama activa ni HEAD a esa rama.

83. ¿Cómo se crea y activa una rama en un paso con la sintaxis del libro?

git checkout -b tema.

84. ¿Qué significa HEAD normalmente?

Una referencia a la rama local actual; puede quedar separado y apuntar directamente a un commit.

85. ¿A qué rama se aplica git merge tema?

A la rama que está activa cuando se ejecuta el comando.

86. ¿Cuándo puede haber fast-forward?

Cuando la punta actual es ancestro de la rama que se integra; basta adelantar la referencia.

87. ¿Qué usa una fusión de tres vías?

Las dos puntas y un ancestro común para construir el resultado integrado.

88. ¿Editar el mismo archivo en dos ramas implica conflicto seguro?

No. Git puede combinar modificaciones compatibles; el conflicto surge cuando no puede resolver automáticamente.

89. ¿Cómo se completa una fusión normal con conflictos?

Resolver contenido, quitar marcas, preparar con add y confirmar la fusión, comprobando antes el resultado.

90. ¿Merge elimina automáticamente la rama integrada?

No. Integración y borrado de la referencia son operaciones separadas.

36. Preguntas: Seguimiento remoto y rebase

Tapá la respuesta e intentá justificarla antes de leer. Cada pregunta admite variantes de opción múltiple, verdadero/falso o respuesta breve.

91. ¿origin/master siempre refleja el servidor en tiempo real?

No. Representa el último estado remoto conocido, actualizado al intercambiar información.

92. ¿Qué significa upstream?

La rama de seguimiento configurada para una rama local, usada por operaciones y comparaciones.

93. ¿Cómo se publica una rama y se configura su seguimiento?

git push -u origin tema.

94. ¿Qué significan ahead y behind?

Commits exclusivos locales y exclusivos de la referencia de seguimiento; dependen de la información disponible localmente.

95. ¿branch -d tema borra tema en el servidor?

No. Para el servidor se usa git push origin --delete tema, con permisos apropiados.

96. ¿Qué diferencia ramas de tema y de larga duración?

Las primeras aíslan una tarea temporal; las segundas sostienen líneas persistentes, por ejemplo niveles de estabilidad.

97. ¿Rebase conserva los identificadores de todos los commits reaplicados?

No. Al cambiar su base/padres genera nuevos objetos e identificadores.

98. ¿Merge y rebase pueden dejar igual contenido final?

Sí, en una integración equivalente, aunque representen una historia diferente.

99. ¿Por qué evitar rebase de trabajo publicado?

Otros pueden basarse en los commits anteriores; reemplazarlos obliga a reconciliar historias y puede causar confusión.

100. ¿Para qué sirve rebase --onto en el ejemplo del capítulo?

Para reaplicar una selección de trabajo sobre otra base, por ejemplo cliente sin incorporar todavía servidor.

37. Práctica: preparar exactamente lo necesario

Ejercicio 1. Predecí el resultado antes de mirar

Existe un commit inicial en el que `nota.txt` contiene A. Se cambia el contenido a B, se ejecuta `git add nota.txt`, se cambia a C y se ejecuta `git commit -m "Actualizar nota"`.

Preguntas: ¿qué contiene el commit nuevo?, ¿qué contiene el archivo de trabajo?, ¿queda algún cambio pendiente?, ¿qué muestran `diff` y `diff --staged`?

Solución: el nuevo commit contiene B; el directorio de trabajo contiene C. Hay un cambio sin preparar entre B y C. `git diff` lo muestra; `git diff --staged` queda vacío porque el índice coincide con el nuevo commit. Para confirmar C hay que volver a preparar y confirmar.

Ejercicio 2. Archivo nuevo y archivo seguido

`programa.cpp` ya está seguido y fue modificado. `manual.txt` es nuevo y untracked. Se ejecuta `git commit -am "Actualizar"`.

Solución: se confirma la modificación de `programa.cpp`, pero no `manual.txt`. Para incluir ambos en un commit normal:

```
git add programa.cpp manual.txt
git commit -m "Actualizar programa y agregar manual"
```

Si el commit `-am` ya se hizo, estos comandos incorporan `manual.txt` en otro commit; no repiten automáticamente la modificación ya registrada.

Ejercicio 3. Separar dos cambios

Se modificaron `a.txt` y `b.txt` y se ejecutó `git add .`, pero se quiere confirmar solo `a.txt`. El repositorio tiene un commit previo.

```
git reset HEAD b.txt
git diff --staged
git commit -m "Actualizar a"
```

Solución: el contenido de `b.txt` se conserva como modificación sin preparar. Usar `checkout -- b.txt` en su lugar descartaría cambios de trabajo y no expresa el mismo objetivo.

Ejercicio 4. Ignorar un archivo ya versionado

Un archivo generado `salida.log` quedó en el historial y debe conservarse localmente sin futuras versiones. Se agrega `salida.log` a `.gitignore`; luego:

```
git rm --cached salida.log
git add .gitignore
git commit -m "Dejar de versionar el archivo generado"
```

Solución: el archivo local permanece, deja de estar en la nueva instantánea y las reglas ayudan a no agregarlo otra vez. Las versiones históricas no desaparecen.

38. Práctica: ramas, fusión y colaboración

Ejercicio 5. Deducir qué punteros se mueven

Historia inicial: A--B, con master activa. Se crea tema, se cambia a ella y se confirma C. Se vuelve a master y se confirma D.

```

      C (tema)
      /
A---B
  \
   D (master, HEAD)

```

Solución: crear tema no cambió HEAD; confirmar C movió solo tema; confirmar D movió solo master. La historia divergió. Estando en master, `git merge tema` normalmente crea una fusión de tres vías, no un fast-forward. Si no se hubiera creado D, master podría avanzar hasta C.

Ejercicio 6. Resolver una diferencia de texto

Master dice «Total con IVA» y tema dice «Importe final» en la misma línea. El resultado acordado es «Importe final con IVA».

Solución: ejecutar merge desde la rama destino, abrir el archivo conflictivo, dejar la frase acordada y borrar los marcadores. Revisar el archivo y ejecutar:

```

git add resumen.txt
git commit -m "Unificar la descripción del importe"
git status

```

Ni la fecha más reciente ni el nombre de la rama deciden por sí solos el contenido correcto.

Ejercicio 7. Trabajo remoto nuevo

Tu master local tiene L, y el servidor agregó R desde el ancestro común. push fue rechazado. Se desea integrar conservando ambas historias.

```

git checkout master
git fetch origin
git log --oneline --graph --decorate --all
git merge origin/master
git push origin master

```

Solución: fetch trae R sin mezclar; merge integra en master, con resolución si hace falta; push publica el resultado. No se propone forzar la publicación porque el objetivo es conservar los dos trabajos.

Ejercicio 8. Rebase local

Tema tiene C encima de B y master avanzó a D desde B. `git checkout tema` y `git rebase master` reconstruyen C como C' después de D. Tema avanza; master sigue en D hasta que se integre. C' y C no tienen el mismo identificador. Esta práctica se plantea para commits aún no compartidos.

39. Práctica: ejecutar un flujo completo

Laboratorio local con la sintaxis de Pro Git

Usá una carpeta de práctica nueva. Las líneas de editar archivos son acciones para realizar en el editor; no son comandos. Elegí tu nombre y correo. No requiere remoto para llegar al tag local.

1. Crear la carpeta, entrar e inicializar Git.

```
git init
git config user.name "Estudiante"
git config user.email "estudiante@example.com"
```

2. Crear README.txt con «Proyecto inicial», prepararlo y confirmar.

```
git add README.txt
git commit -m "Crear proyecto inicial"
git branch
```

3. Identificar el nombre de la rama principal. Si no es master, usar su nombre en los pasos siguientes. Crear la rama mejora, agregar una línea al README y confirmar.

```
git checkout -b mejora
git add README.txt
git commit -m "Documentar una mejora"
```

4. Volver a la rama principal e integrar. Si no hubo nuevos commits allí, se espera fast-forward.

```
git checkout master
git merge mejora
git log --oneline --graph --decorate --all
git branch -d mejora
git tag -a v1.0 -m "Versión de práctica"
git show v1.0
```

Resultado esperado: dos commits, el segundo accesible desde la rama principal, la rama mejora eliminada y el tag apuntando al segundo commit. Eliminar mejora no elimina ese trabajo ya integrado.

Extensión con remoto existente y autorizado: registrar su URL con `git remote add origin URL`, publicar con `git push -u origin master` y enviar la etiqueta con `git push origin v1.0`. La URL es un marcador que debe sustituirse; el servidor tiene que existir y permitir escritura.

Comprobación de aprendizaje: explicá después de cada comando qué cambió: archivo, índice, historial, referencia local o referencia remota. Si no podés decirlo, volvé a las tres áreas de Git.

40. Práctica conceptual de ingeniería

Caso 1. Reservas con requisitos inciertos

Una organización quiere una aplicación de reservas, pero los usuarios no tienen claras sus pantallas ni reglas de modificación.

Respuesta modelo: usar prototipos para descubrir y validar necesidades y desarrollo incremental para construir funciones prioritarias. Entregar operativamente por partes si esas partes pueden usarse de forma útil. No elegir cascada rígida solo porque facilita documentación: los requisitos inciertos aumentan el retrabajo.

Caso 2. Controlador crítico

Se debe controlar un dispositivo cuya actuación incorrecta podría producir un accidente.

Respuesta modelo: enfatizar confiabilidad, seguridad, requisitos completos y análisis de interacciones. Un proceso planificado y verificación rigurosa resulta justificable. Los prototipos pueden ayudar a explorar riesgos, pero uno desechable no debe convertirse sin más en el controlador de producción. No existe una elección automática basada únicamente en la palabra «crítico».

Caso 3. Sistema administrativo estándar

Existen productos que cubren casi todas las necesidades de una empresa.

Respuesta modelo: evaluar reutilización/COTS: identificar componentes, revisar requisitos negociables, diseñar integración y validar. Ventajas: menos desarrollo; riesgos: ajuste imperfecto y dependencia de la evolución del proveedor.

Caso 4. Cumple el documento, pero nadie lo puede usar

El sistema implementa cada requisito escrito, pero el flujo hace que los usuarios no puedan completar su trabajo.

Respuesta modelo: puede haber conformidad con la especificación y fallar la validación de necesidades o la aceptabilidad. Hay que revisar requisitos y trabajo real; no basta afirmar que pasó las pruebas técnicas.

Caso 5. Cambio solicitado por el cliente

El cliente pide una nueva función y el programador modifica directamente una versión vieja.

Respuesta modelo: la solicitud necesita registro y análisis de impacto; deben identificarse la configuración y versión correctas. SCM evita perder trazabilidad y entregar combinaciones equivocadas. Git conserva versiones, pero el equipo debe establecer el proceso de decisión y entrega.

Caso 6. Presión para ocultar un fallo

Una persona recibe presión para dar por validado un sistema aunque conoce una falla relevante.

Respuesta modelo: identificar riesgos y afectados, mantener integridad de la evidencia y comunicar el problema por los canales adecuados. Considerar interés público, producto, juicio y responsabilidades con cliente y empleador; no reducir la ética a obedecer una orden ni inventar resultados de pruebas.

41. Repaso de última vuelta

Veinte asociaciones que conviene dominar

- Software: programas + documentación + configuración.
- Ingeniería de software: producción completa bajo restricciones.
- 1968: conferencia vinculada a la crisis del software.
- Actividades fundamentales: especificación, desarrollo, validación, evolución.
- Especificación: servicios y restricciones; validación: conformidad y necesidades.
- Apoyo del formulario: proyectos, configuración, calidad.
- Cascada: requisitos estables y fases; incremental: versiones y adaptación.
- Prototipo: aprender; entrega incremental: usar partes reales.
- Espiral: riesgos; RUP: concepción, elaboración, construcción, transición.
- SCM: controlar artefactos y cambios; Git: control de versiones.
- Git: distribuido; commits y ramas pueden ser offline.
- init crea repositorio; clone copia uno existente.
- add prepara; commit confirma; push publica.
- commit normal guarda índice; commit -a no incorpora untracked por sí solo.
- status informa; diff compara; log recorre historia; show detalla.
- .gitignore no retira seguimiento existente ni borra el pasado.
- branch crea; checkout cambia; checkout -b hace ambas cosas.
- merge integra EN la rama activa; fast-forward no necesita un commit de fusión.
- fetch obtiene; pull obtiene e integra; origin/master es conocimiento local del remoto.
- rebase reaplica y cambia identificadores; tag no avanza automáticamente.

Cómo responder un múltiple opción

Leé primero la consigna completa. Identificá si pide la correcta, la incorrecta o la mejor alternativa. Ubicá el concepto antes de leer las letras. Descartá opciones que confundan áreas, actividades o alcances. Si una alternativa usa «siempre» o «únicamente», comprobá si hay un contraejemplo del capítulo.

Autoevaluación

Podés considerar un tema aprendido si explicás la respuesta sin verla, das un ejemplo nuevo y justificás por qué una alternativa parecida sería incorrecta. En Git, además, tenés que predecir el estado después de una secuencia de comandos.

Antes del escrito: revisá especialmente las preguntas 3, 4 y 5 de procesos y las de identidad, CLI, distribución y SCM de tus capturas. Son los puntos donde aparecen errores que se pueden corregir comprendiendo una distinción breve.

42. Fuentes y cobertura

Libros aportados

Ian Sommerville. Ingeniería de software, 9.ª edición, traducción española, 2011. Capítulo 1: pp. 3-26; capítulo 2: pp. 27-55. En el archivo aportado corresponden a páginas PDF 21-44 y 45-73. El complemento de SCM procede de pp. 682-684, páginas PDF 700-702.

Scott Chacon y Ben Straub. Pro Git, segunda edición, traducción española. El archivo identifica la versión 2.1.22-4-g2264d13, de 2021-08-14. Capítulo 1: pp. 9-23; capítulo 2: pp. 24-58; capítulo 3: pp. 59-99. En el PDF corresponden a páginas 14-28, 29-63 y 64-104.

Sitio de referencia indicado en tu programa: <https://git-scm.com/book/en/v2>. Esta guía se basó en los archivos aportados; no reemplaza sus ejemplos extensos ni reproduce los capítulos completos.

Control de cobertura del resumen

- Sommerville 1.1: software profesional, productos, atributos, disciplinas, diversidad, fundamentos y Web.
- Sommerville 1.2: responsabilidad profesional y ocho principios éticos.
- Sommerville 1.3: bomba de insulina, pacientes y estación meteorológica.
- Sommerville 2.1: cascada, incremental, reutilización y menciones de desarrollo formal.
- Sommerville 2.2: requisitos, diseño, implementación, validación y evolución.
- Sommerville 2.3: anticipación/tolerancia al cambio, prototipos, entrega incremental y espiral.
- Sommerville 2.4: perspectivas, fases, flujos y prácticas de RUP.
- Pro Git 1: control de versiones, historia, fundamentos, CLI, instalación, configuración y ayuda.
- Pro Git 2: repositorios, seguimiento, diferencias, commits, archivos, log, deshacer, remotos, tags y alias.
- Pro Git 3: ramas, HEAD, merge, conflictos, gestión y flujos, seguimiento remoto y rebase.

Ilustraciones y preguntas

Se incluyeron 17 recortes de figuras originales con su número, página y explicación. Las figuras de Sommerville mantienen su atribución al libro; la espiral conserva la mención a IEEE que figura allí. Pro Git declara licencia CC BY-NC-SA 3.0; sus recortes se atribuyen a sus autores y a la segunda edición.

Las 24 respuestas se vinculan a tus cuatro capturas. Las 100 preguntas adicionales y los casos de práctica son material de estudio elaborado para esta guía. Las ambigüedades observadas se señalan expresamente: SCM, alcance de «todos» en CLI y formulaciones simplificadas de algunos comandos.

Límite del alcance: no se incluyeron como temario adicional los capítulos de Scrum/XP, requisitos avanzados, servidores Git o herramientas avanzadas. Solo se agregaron aclaraciones breves necesarias para comprender las preguntas y ejecutar los flujos básicos.